03 - PostgreSQL Setup

The database, explained from zero, using this project's actual settings.

What is this?

PostgreSQL is the database this project uses. A database is a program that stores information on disk in an organised way and hands it back when asked.

Like: the steel almirah of ledgers in the shop's back office. Switch off the lights, come back tomorrow, every entry is still there.

> **IMPORTANT**
> This project uses PostgreSQL, not MySQL
> An earlier draft of this project used MySQL. It was migrated to PostgreSQL before any deployment. If you find MySQL mentioned anywhere outside a historical note, that is a bug - report it. `registry/static_check.py` fails the build if MySQL syntax reappears in a migration.

The words you need

Word	Simple meaning	In this project
Server	The database program, running and waiting	PostgreSQL 16 in Docker
Database	One named collection of tables	`hardware_erp`
Schema	A folder for tables inside a database	`public` (the default)
Table	A grid, like one sheet in a ledger	`app_user`, `role`, `permission`
Column	One vertical field in the grid	`mobile_no`, `full_name`
Row	One horizontal record	One employee
User	A database login	`hardware_erp`
Password	That login's password	Set by you
Host	Which machine the server is on	`localhost`
Port	Which door on that machine	`5432`
Connection	An open line from app to database	Managed by the pool

How the pieces connect

```
SPRING BOOT APPLICATION
     |
     |  "find the user with mobile 9876543210"
     v
POSTGRESQL JDBC DRIVER      a translator: Java calls -> database language
     |
     |  network, localhost:5432
     v
POSTGRESQL SERVER           the running database program
     |
     v
DATABASE  hardware_erp
     |
     v
TABLES    app_user, role, permission, refresh_token,
          password_reset_token, role_permission, security_audit_log
```

The **driver** is the piece people forget. Java cannot speak to PostgreSQL directly. `pom.xml` includes `org.postgresql:postgresql`, which is the translator.

This project's actual settings

Read from `backend/src/main/resources/application.yml` and `docker-compose.yml`:

Setting	Value	Environment variable
Host	`localhost`	`DB_HOST`
Port	`5432`	`DB_PORT`
Database name	`hardware_erp`	`DB_NAME`
Username	`hardware_erp`	`DB_USER`
Password	(you choose)	`DB_PASSWORD`
Driver	`org.postgresql.Driver`	fixed
Full JDBC URL	`jdbc:postgresql://localhost:5432/hardware_erp`	built from the above

> **IMPORTANT**
> The database user is not "postgres"
> It is `hardware_erp`. Using `postgres` - the superuser - would mean the application could drop any database on the server. It only needs its own.

Option A - Docker (recommended)

This is the fastest path and the one the project is set up for.

SETUP

Before you start

Docker Desktop must be installed and running. Check with `docker --version`.

RUN THIS

Start PostgreSQL

```
cd hardware-erp
docker compose up -d
```

WHAT YOU SHOULD SEE

Expected output

```
[+] Running 2/2
 Network hardware-erp_default    Created
 Container hardware-erp-postgres Healthy
```

The word **Healthy** matters. It means the healthcheck ran `pg_isready` against the real database and user, not just that the container started.

RUN THIS

Confirm it is listening

```
docker compose ps
```

WHAT YOU SHOULD SEE

Expected

```
NAME                    STATUS                    PORTS
hardware-erp-postgres   Up 30 seconds (healthy)   0.0.0.0:5432->5432/tcp
```

What `docker-compose.yml` sets up for you:

- PostgreSQL **16-alpine**
- Database `hardware_erp`, user `hardware_erp`
- Encoding **UTF8**, locale `C.UTF-8`
- Timezone `Asia/Kolkata`
- A named volume so your data survives `docker compose down`

IMPORTANT

Set a password before you start it

Copy `.env.example` to `.env` in the project root and set `DB_PASSWORD`. If you skip this, the compose file falls back to `hardware_erp` as the password, which is fine locally and unacceptable anywhere else.

Option B - Install PostgreSQL directly

Use this if you cannot run Docker.

1. Download PostgreSQL 16 from postgresql.org/download
2. Run the installer. When it asks for a password, that is for the `postgres` superuser - write it down
3. Keep port **5432**
4. Let it install **pgAdmin 4** as well

Then create the database and the application user:

RUN THIS

Open a psql prompt as the superuser

```
psql -U postgres
```

RUN THIS

Create the user and database

```
CREATE USER hardware_erp WITH PASSWORD 'your-password-here';
CREATE DATABASE hardware_erp OWNER hardware_erp ENCODING 'UTF8';
\q
```

WHY

Why ENCODING UTF8
So the database can store any character - customer names in Tamil, the rupee sign, anything. The Docker setup does this for you.

Using pgAdmin to look inside

pgAdmin is a browser-based tool for browsing the database by clicking.

1. Open pgAdmin
2. Right-click **Servers**, choose **Register > Server**
3. **General** tab: Name it `Hardware ERP Local`
4. **Connection** tab:

Field	Value
Host name/address	`localhost`
Port	`5432`
Maintenance database	`hardware_erp`
Username	`hardware_erp`
Password	whatever you set

5. Tick **Save password**, click **Save**

WHAT YOU SHOULD SEE

What you should see
The server appears in the left tree. Expand
**Servers > Hardware ERP Local > Databases > hardware_erp > Schemas > public >
Tables **and you should find seven tables -** but only after the backend has
run once**, because Flyway creates them. Before that the list is empty, and
that is correct.

Connecting Spring Boot to PostgreSQL

You do not have to write any connection code. It is all in

`application.yml`:

```
spring:
  datasource:
    url: jdbc:postgresql://${DB_HOST:localhost}:${DB_PORT:5432}/${DB_NAME:hardware_erp}
    username: ${DB_USER:hardware_erp}
    password: ${DB_PASSWORD:}
    driver-class-name: org.postgresql.Driver
```

The `${NAME:default}` syntax means "use the environment variable `NAME`, and if

it is not set, use this default".

RUN THIS

Start the backend

```
cd backend
./mvnw spring-boot:run
```

WHAT YOU SHOULD SEE

Signs the connection worked
In the console you should see, in this order:

```
HikariPool-1 - Starting...
HikariPool-1 - Added connection org.postgresql.jdbc.PgConnection@...
Flyway Community Edition ... by Redgate
Successfully validated 2 migrations
Started HardwareErpApplication in 6.2 seconds
```

`HikariPool` is the connection pool. Seeing it add a connection means Java
reached PostgreSQL successfully.

Verify the whole chain

VERIFY

Four checks, from database up to application

1. The server is alive

```
docker exec hardware-erp-postgres pg_isready -U hardware_erp -d hardware_erp
```

Expect: `/var/run/postgresql:5432 - accepting connections`

2. The tables exist

```
docker exec -it hardware-erp-postgres psql -U hardware_erp -d hardware_erp -c "\dt"
```

Expect seven tables plus `flyway_schema_history`.

3. The seed data loaded

```
docker exec -it hardware-erp-postgres psql -U hardware_erp -d hardware_erp -c "SELECT mob
ile_no, full_name, status FROM app_user ORDER BY user_id LIMIT 5;"
```

Expect Saravanan Murugan on 9876543210 and four more rows.

4. The application can use it

```
curl http://localhost:8080/api/actuator/health
```

Expect `{"status":"UP"}`.

If it fails

IF IT FAILS

"Connection to localhost:5432 refused"
PostgreSQL is not running. `docker compose up -d`, then `docker compose ps` and wait for `healthy`.

IF IT FAILS

"password authentication failed for user hardware_erp"
The password in your `.env` does not match the one the database was created with. Because the data lives in a Docker volume, changing `.env` afterwards does not change the stored password. Either use the original password, or wipe and recreate:

```
docker compose down -v
docker compose up -d
```

`-v` deletes the volume and all its data. Only do this in development.

IF IT FAILS

"database hardware_erp does not exist"
You are on Option B and skipped the `CREATE DATABASE` step, or you are connecting to a different PostgreSQL that was already using port 5432.

IF IT FAILS

"port is already allocated"
Something else already holds 5432 - very often a PostgreSQL you installed earlier. Either stop it, or set `DB_PORT=5433` in `.env` and use that port everywhere.

Document 17 covers these in more detail along with everything else.